



✓ You already completed this chapter.

3.2 Optional and Default Parameters

Not completed (x)

Why Optional and Default Parameters Matter (Extra Documentation)

“ No video for this lesson, but plenty of code examples and explanations! ”

Real-World Analogy: Think of ordering at a coffee shop. You can order just "coffee" (required parameter), but you might also specify "size" (optional - defaults to medium) or "milk type" (optional - defaults to regular milk). The barista knows how to handle both detailed orders and simple ones.

Similarly, TypeScript functions can have:

Required parameters: Must be provided (like "coffee")

Optional parameters: Can be omitted (like "size")

Default parameters: Have fallback values (like "medium" for size)

1 Optional Parameters

Optional parameters are marked with a `?` after the parameter name. They can be omitted when calling the function.

Basic Optional Parameters

```
1 // Function with optional parameter
2 function greet(name: string, greeting?: string): string {
3     if (greeting) { // if the optional parameter is provided
4         return `${greeting}, ${name}!`;
5     }
6     return `Hello, ${name}!`;
7 }
```

```
11 console.log(greet("Bob", "Good morning")); // "Good morning, Bob!"
```

Type of Optional Parameters

Optional parameters can have either a type or be `undefined` if not provided.

```
1 function displayPrice(product: string, discount?: number): void {
2   // discount has type: number | undefined
3
4   if (discount !== undefined) {
5     console.log(`${product}: ${100 - discount}`);
6   } else {
7     console.log(`${product}: $100`);
8   }
9 }
10
11 displayPrice("Widget"); // "Widget: $100"
12 displayPrice("Gadget", 20); // "Gadget: $80"
```

Multiple Optional Parameters

```
1 function createUser(
2   username: string,
3   email?: string,
4   age?: number,
5   bio?: string
6 ): {username: string, email?: string, age?: number, bio?: string} {
7   const user: {username: string, email?: string, age?: number, bio?: string} =
8
9   if (email !== undefined) user.email = email;
10  if (age !== undefined) user.age = age;
11  if (bio !== undefined) user.bio = bio;
12
13  return user;
14 }
15
16 // Various usage patterns
```

```
20 const user4 = createUser("dave", "dave@example.com", 25, "Software developer",
```

2 Default Parameters

Default parameters have a fallback value that's used when the argument is `undefined` or omitted.

Basic Default Parameters

```
1 function calculatePrice(price: number, tax: number = 0.1): number {
2   return price + (price * tax);
3 }
4
5 console.log(calculatePrice(100));           // 110 (uses default 0.1)
6 console.log(calculatePrice(100, 0.2));     // 120 (uses provided 0.2)
7 console.log(calculatePrice(100, 0));       // 100 (uses provided 0)
```

Default Parameters with Complex values

```
1 function createConfig(
2   name: string,
3   options = { debug: false, maxRetries: 3, timeout: 5000 }
4 ) {
5   return {
6     name,
7     debug: options.debug,
8     maxRetries: options.maxRetries,
9     timeout: options.timeout
10  };
11 }
12
13 // Uses all defaults
14 const config1 = createConfig("app1");
15
16 // Override entire options object
17 const config2 = createConfig("app2", {
```

```
21 });
```

Default Parameters with Functions

```
1 function processData(
2   data: string[],
3   transform: (item: string) => string = (item) => item.toUpperCase()
4 ): string[] {
5   return data.map(transform);
6 }
7
8 const names = ["Alice", "Bob", "Charlie"];
9
10 // Uses default transform (uppercase)
11 console.log(processData(names));
12 // ["ALICE", "BOB", "CHARLIE"]
13
14 // Uses custom transform
15 console.log(processData(names, (item) => item.toLowerCase()));
16 // ["alice", "bob", "charlie"]
```

3 Parameter Ordering Rules

TypeScript enforces specific ordering rules for parameters:

Rule 1: Required Before Optional

```
1 // ✅ Correct: Required parameters first
2 function correct(required: string, optional?: number): void {
3   console.log(required, optional);
4 }
5
6 // ❌ Error: Optional parameters must come after required ones
7 function incorrect(optional?: string, required: number): void {
8   //           ^^^^^^^
9   // Error: A required parameter cannot follow an optional parameter
```

Rule 2: Default Parameters Can Be Anywhere

Default parameters don't need to be at the end:

```
1 function createAccount(  
2   username: string,  
3   role: string = "user", // Default in middle is OK  
4   email: string  
5 ): object {  
6   return { username, role, email };  
7 }  
8  
9 // Must provide username and email, role is optional  
10 createAccount("alice", undefined, "alice@example.com"); // here role is default  
11 createAccount("bob", "admin", "bob@example.com");
```



Best Practice: Put Optional/Default at End

While TypeScript allows default parameters anywhere, it's cleaner to put them at the end:

```
1 // ✅ Best practice: Optional/default parameters at end  
2 function betterCreateAccount(  
3   username: string,  
4   email: string,  
5   role: string = "user"  
6 ): object {  
7   return { username, email, role };  
8 }  
9  
10 // Much cleaner to call  
11 betterCreateAccount("alice", "alice@example.com");  
12 betterCreateAccount("bob", "bob@example.com", "admin");
```



Destructuring with Optional Parameters

```
1  type RequestOptions = {
2    method?: string;
3    headers?: Record<string, string>;
4    timeout?: number;
5  }
6
7  function makeRequest(
8    url: string,
9    {
10     method = "GET",
11     headers = {},
12     timeout = 5000
13   }: RequestOptions = {} // Note: entire object is optional
14 ): Promise<{ status: number }> {
15   console.log(`${method} ${url}`);
16   console.log("Headers:", headers);
17   console.log("Timeout:", timeout);
18
19   // Simulated fetch logic
20   return Promise.resolve({ status: 200 });
21 }
22
23 // All these calls are valid
24 makeRequest("https://api.example.com/data");
25
26 makeRequest("https://api.example.com/data", {
27   method: "POST" // other properties use defaults
28 });
29
30 makeRequest("https://api.example.com/data", {
31   method: "POST",
32   headers: { "Content-Type": "application/json" },
33   timeout: 10000
34 });
```

Array Destructuring with Defaults

```
4   return { x, y, z };
5 }
6
7 console.log(parseCoordinates());           // { x: 0, y: 0, z: 0 }
8 console.log(parseCoordinates([10]));       // { x: 10, y: 0, z: 0 }
9 console.log(parseCoordinates([10, 20]));   // { x: 10, y: 20, z: 0 }
10 console.log(parseCoordinates([10, 20, 30])); // { x: 10, y: 20, z: 30 }
```

5 Handling Undefined vs Null

Understanding how optional and default parameters handle `undefined` and `null`:

Default Parameters Only Trigger on Undefined

```
1 function setDefault(value: string = "default"): string {
2   return value;
3 }
4
5 console.log(setDefault());           // "default" (undefined)
6 console.log(setDefault(undefined)); // "default" (explicitly undefined)
7 console.log(setDefault(null as any)); // null (not undefined, so default not us
8 console.log(setDefault(""));        // "" (empty string, not undefined)
```



Explicit Undefined Check

```
1 function processValue(value?: string): string {
2   // Check explicitly for undefined
3   if (value === undefined) {
4     return "No value provided";
5   }
6
7   if (value === null) {
8     return "Null value provided";
9   }
10 }
```

```
14
15     return `Value: ${value}`;
16 }
17
18 console.log(processValue());           // "No value provided"
19 console.log(processValue(undefined)); // "No value provided"
20 console.log(processValue(null as any)); // "Null value provided" // very bad p
21 console.log(processValue(""));        // "Empty string provided"
22 console.log(processValue("hello"));    // "Value: hello"
```

Best Practices

1. Use Optional for Truly Optional Values

```
1 // ✅ Good: Parameter is genuinely optional
2 function sendEmail(to: string, subject: string, cc?: string): void {
3     // cc is optional - not all emails need CC
4 }
5
6 // ❌ Avoid: Required data marked as optional
7 function createUser(username?: string, email?: string): void {
8     // These should probably be required
9 }
```

2. Provide Sensible Defaults

```
1 // ✅ Good: Meaningful defaults
2 function retry(
3     fn: () => Promise<any>,
4     maxAttempts: number = 3,
5     delayMs: number = 1000
6 ): Promise<any> {
7     // Defaults make sense for most use cases
8 }
9
```



```
13   width: number = 1920, // too specific
14   height: number = 1080 // Not flexible enough
15 ): void {
16   // Better to make these required or use more generic defaults
17 }
```



Practice Exercise

Create a function called `formatCurrency` with the following signature:

```
1  function formatCurrency(
2    amount: number,
3    currency?: string,
4    locale?: string,
5    options?: {
6      minimumFractionDigits?: number;
7      maximumFractionDigits?: number;
8      useGrouping?: boolean;
9    }
10 ): string {
11   // Your implementation here
12 }
```

Requirements:

Default currency should be "USD"

Default locale should be "en-US"

Default options should be: `minimumFractionDigits: 2`, `maximumFractionDigits: 2`, `useGrouping: true`

Use `Intl.NumberFormat` for formatting (google it if you're not familiar)

Handle all parameter combinations

Test cases:

```
1  console.log(formatCurrency(1234.56));
2  // Expected: "$1,234.56"
3
4  console.log(formatCurrency(1234.56, "EUR", "de-DE"));
```

```
8     minimumFractionDigits: 0,  
9     useGrouping: false  
10  }));  
11  // Expected: "$1234.5"
```

Solution:

```
1  function formatCurrency(  
2    amount: number,  
3    currency: string = "USD",  
4    locale: string = "en-US",  
5    options: {  
6      minimumFractionDigits?: number;  
7      maximumFractionDigits?: number;  
8      useGrouping?: boolean;  
9    } = {}  
10 ): string {  
11   const {  
12     minimumFractionDigits = 2,  
13     maximumFractionDigits = 2,  
14     useGrouping = true  
15   } = options;  
16  
17   const formatter = new Intl.NumberFormat(locale, {  
18     style: "currency",  
19     currency,  
20     minimumFractionDigits,  
21     maximumFractionDigits,  
22     useGrouping  
23   });  
24  
25   return formatter.format(amount);  
26 }  
27  
28 // Test cases  
29 console.log(formatCurrency(1234.56));  
30 // "$1,234.56"
```

```
34
35 console.log(formatCurrency(1234.5, "USD", "en-US", {
36   minimumFractionDigits: 0,
37   useGrouping: false
38 }));
39 // "$1234.5"
40
41 console.log(formatCurrency(9999.999, "GBP", "en-GB", {
42   maximumFractionDigits: 1
43 }));
44 // "£10,000.0"
```

Not completed (x)